

FIELD GUIDE

Agentic AI, MCP & Amazon Bedrock AgentCore

From a plain LLM call to production agents on AWS — intuition first, then the engineering. Concepts → Coding → AgentCore → End-to-end examples.

Prepared for Kaniz

A self-paced tutor companion — read it through, then bring your doubts back.

How to read this guide

Every section opens with a kid-simple analogy, then goes deep for the engineer. You can read straight through, or jump to a phase using the contents on the next page.

Blue boxes = key ideas. Green boxes = tips/best practice. Orange boxes = pitfalls to avoid.

Dark boxes = code or config you can actually run.

Contents

Part 0 · The two questions you couldn't answer (start here)

Phase 1 · Concepts & mental models

- 1.1 Plain LLM call vs an agent
- 1.2 The agent loop: think → act → observe
- 1.3 Tools — how a brain gets hands
- 1.4 Memory — short-term and long-term
- 1.5 Planning, reflection and feedback loops
- 1.6 Agentic frameworks compared (LangGraph, CrewAI, Strands, LlamaIndex)
- 1.7 MCP — the universal plug for tools

Phase 2 · Coding knowledge (hands-on)

- 2.1 A plain Bedrock model call
- 2.2 Adding tool calling (the Converse API)
- 2.3 A minimal agent loop from scratch
- 2.4 Build a tiny MCP server + client
- 2.5 Folder structure & tool-design rules

Phase 3 · AWS Bedrock AgentCore

- 3.1 Where plain Bedrock fits
- 3.2 Mapping agent concepts onto AWS building blocks
- 3.3 AgentCore at a glance — the modular services
- 3.4 Runtime — where agents actually execute
- 3.5 Gateway — turn APIs into MCP tools
- 3.6 Memory · 3.7 Identity · 3.8 Observability, Evaluations, Policy
- 3.9 How AgentCore hosts frameworks & MCP
- 3.10 MCP + AgentCore + Claude working together

Phase 4 · End-to-end examples

- A. Customer support agent
- B. Internal ops assistant (with Step Functions)
- C. Coding / research assistant over MCP
- D. Document-intelligence agent (RAG + Knowledge Bases)

Recipes · Copy-and-keep checklists

Pre-flight · Questions to ask before you ship

Best practices & pitfalls · Glossary

Part 0 · The two questions you couldn't answer

You said you didn't know the answers to the two checks. Perfect — here they are, in plain words, because everything else in this guide hangs off them.

Q1. What single thing turns a plain LLM call into an agent?

The one-line answer

A **loop with the ability to act**. A plain LLM call is one-shot: text in, text out. An agent puts that same model inside a cycle where, on each turn, it can **choose to call a tool**, see the result, and decide what to do next — repeating until the goal is met.
Brain alone = LLM. Brain + hands + the think-act-observe loop = agent.

A useful test: if the system can *decide on its own* to take an action (look something up, call an API, run code), look at what happened, and change its next step because of it — that's an agent. If it just produces one answer with no ability to act or iterate, it's a plain model call, no matter how clever the prompt.

Q2. Why does a standard like MCP matter — what pain does it remove?

The one-line answer

MCP removes the **N×M integration explosion**. Without a standard, every agent (or app) needs custom glue code for every tool: 5 agents × 10 tools = 50 bespoke integrations to write and maintain.
With MCP, a tool is wrapped **once** behind a standard interface (an MCP server). Any MCP-speaking agent can discover and call it with zero custom code — 5 + 10 instead of 5 × 10. It's USB-C for AI tools.

Concretely, MCP standardises four things so nobody has to reinvent them per tool: how tools advertise themselves (discovery), how arguments and results are shaped (schemas), how the call is transported (stdio or HTTP), and how auth is negotiated. Build to the standard once and the tool is portable across Claude Desktop, an IDE assistant, and a cloud runtime alike.

Phase 1 · Concepts & mental models

Goal of this phase: a rock-solid intuition for what an agent is, what the pieces are called, and where MCP fits — before we touch any code.

1.1 Plain LLM call vs an agent

Kid version. A plain LLM call is like sliding a note through a mail slot to a very smart friend locked in a room. One note in, one note back — from memory only. They can't stand up, can't look anything up, can't *do* anything. An agent is that same friend, but now standing in a workshop with tools on the wall and a notepad, free to walk around, try things, look at results, and try again.

Engineer version. A plain call is a single, stateless request to a model: you send messages, you get back one completion. An agent wraps that call in a control loop and gives the model a set of callable tools. On each turn the model may return a normal answer or a structured request to invoke a tool. Your code runs the tool, feeds the result back into the context, and calls the model again. The model is the reasoning engine; the agent is the engine plus tools plus the orchestration loop plus (usually) memory.

Property	Plain LLM call	Agent
Shape	One request & one response	Loop of many model calls + tool calls
Can take actions?	No	Yes — calls tools / APIs / code
State	Stateless	Carries context; often persists memory
Stops when	Response returned	Goal met or stop condition hit
Failure mode	Wrong answer	Wrong answer, or wrong/looping actions

1.2 The agent loop: think → act → observe

This loop is the heart of every agent framework. Everything fancier is a variation on it.

```
goal -> [ THINK ]   model decides the next step
        [ ACT ]     if it chose a tool: run the tool
        [ OBSERVE] feed the tool's result back in
        |__ repeat until the model returns a final answer
```

Mechanically, the model signals "I want to act" by emitting a **tool-use** block: a small structured object naming a tool and its arguments, e.g. `get_weather(city="Pune")`. Your runtime executes that function and returns a **tool-result** block. The model reads it and either acts again or answers. This is the ReAct pattern (Reason + Act) you'll see referenced everywhere.

Pitfall: the runaway loop

Because the agent decides its own next step, it can loop forever, repeat a failing call, or rack up cost. Always cap iterations (e.g. max 8 steps), set per-tool timeouts, and have a fallback answer when the budget is hit. We design for this properly in Phase 2.

1.3 Tools — how a brain gets hands

Kid version. Tools are the drill, the saw, the tape measure on the workshop wall. The brain picks the right one for the step it's on.

Engineer version. A tool is just a function the model is allowed to call, described to the model by a **name**, a **natural-language description**, and a **JSON schema** for its inputs. The description is part of your prompt engineering: the model chooses tools based on it, so vague descriptions cause wrong tool choices. Good tools are small, single-purpose, and return structured, predictable output.

- **Read tools** fetch data (search a KB, query a DB, call a GET endpoint) — safe to retry.
- **Write tools** change the world (issue a refund, send an email, create a ticket) — must be designed carefully.
- **Compute tools** run code or do math in a sandbox (e.g. a code interpreter).

1.4 Memory — short-term and long-term

Kid version. Short-term memory is the sticky note on the agent's desk for the job it's doing right now. Long-term memory is the filing cabinet it keeps between visits.

Engineer version. The model itself is stateless between calls; "memory" is something you build around it.

Type	What it holds	Typical implementation
Short-term	The current conversation / task context	The running message list in the loop
Long-term	Facts, preferences, summaries across sessions	Vector store / DB, retrieved per turn
Episodic	Past interactions the agent can learn from	Stored sessions, summarised + indexed

The skill is deciding *what* to remember and *what* to inject back into context, because context windows and tokens cost money. AgentCore's Memory service (Phase 3) productises exactly this split.

1.5 Planning, reflection and feedback loops

Simple agents react step-by-step. More capable ones add structure:

- **Planning** — the agent first drafts a multi-step plan, then executes it, re-planning if reality diverges.
- **Reflection / self-critique** — the agent reviews its own output or a tool result and decides whether to redo it.

- **Multi-agent** — several specialised agents (a researcher, a writer, a checker) collaborate, often coordinated by a supervisor.
- **Human-in-the-loop** — the agent pauses for approval before a risky write action.

Mental model to keep

User → Agent (brain + loop) → Tools (via Gateway/MCP) → External systems → back to the agent, with Memory read/written along the way and every step emitted to Logs/Traces.
Hold this picture; all of Phase 3 is just AWS-flavoured boxes for each arrow.

1.6 Agentic frameworks compared

A framework gives you the loop, tool plumbing, memory hooks and state management so you don't hand-roll them. They differ in philosophy. AgentCore runs *all* of these, so the choice is about developer ergonomics, not lock-in.

Framework	Best for	Flavour
Strands Agents	AWS-native agents, fastest path to Bedrock + AgentCore	Lightweight, model-driven loop; AWS-maintained
LangGraph	Complex, branching, stateful workflows with cycles	Explicit graph of nodes/edges; maximum control
CrewAI	Multi-agent teams with roles collaborating	Role-and-task abstraction; quick to express crews
LlamaIndex	RAG-heavy agents over your documents/data	Strong data-ingestion + retrieval primitives

Which to pick when

Starting on AWS and want minimum ceremony → **Strands**. Need precise control over a stateful, looping workflow → **LangGraph**. Modelling a team of cooperating personas → **CrewAI**. The agent's whole job is reasoning over your corpus → **LlamaIndex**. You can switch later — AgentCore Runtime is framework-agnostic.

1.7 MCP — the universal plug for tools

Kid version. Imagine every tool had a different weird plug, so the robot wasted its life hunting for adapters. MCP is USB-C: one standard plug, any tool snaps into any robot.

Engineer version. The Model Context Protocol is an open standard (think "LSP, but for AI tools and data") with three roles:

- **MCP server** — wraps a capability and exposes it in a standard way. It advertises *tools* (callable functions), and can also expose *resources* (readable data) and *prompts* (reusable templates).
- **MCP client** — lives inside the host (Claude Desktop, an IDE, a cloud runtime). It connects to servers, lists what they offer, and forwards calls.
- **Host / agent** — the LLM-driven application that decides which tool to call and when.

Transport is either **stdio** (the server runs as a local subprocess — great for desktop/IDE) or **streamable HTTP** (the server is a remote endpoint — how cloud services like AgentCore Gateway expose tools). The payloads are JSON-RPC under the hood, but you rarely touch that directly.

Why this is a big deal for AWS

AgentCore Gateway (Phase 3) is essentially a managed, secured MCP server: you point it at your Lambda functions or OpenAPI specs and it publishes them as MCP tools with auth, so any MCP client — your agent in Runtime, or Claude on your laptop — can call your enterprise APIs without bespoke glue.

Phase 2 · Coding knowledge (hands-on)

Now we earn the intuition with real Python. We build up in the same order as the mental model: a bare model call → tool calling → a full agent loop → an MCP server. Snippets are deliberately minimal and runnable; production hardening comes in Phases 3–4.

2.1 A plain Bedrock model call

This is the "note through the mail slot" — one request, one answer. We use the Bedrock **Converse** API, which gives a single uniform shape across all models (Claude, Nova, Llama...).

```
import boto3

brt = boto3.client("bedrock-runtime", region_name="us-east-1")

resp = brt.converse(
    modelId="anthropic.claude-3-5-sonnet-20240620-v1:0",
    messages=[{"role": "user",
                "content": [{"text": "Explain MCP in one sentence."}]}],
    inferenceConfig={"maxTokens": 300, "temperature": 0.2},
)
print(resp["output"]["message"]["content"][0]["text"])
```

What to notice

No tools, no loop, no memory — this is the baseline. Everything an agent adds is layered on top of this exact call. The model is reached via an IAM-authenticated AWS API; there is no separate API key to manage.

2.2 Adding tool calling (the Converse API)

We declare tools with a name, description, and JSON schema. The model can now *request* a call instead of answering. Note we don't execute anything yet — we just see the request come back.


```

tool_config = {
  "tools": [{
    "toolSpec": {
      "name": "get_order_status",
      "description": "Look up the current status of a customer order by id.",
      "inputSchema": {"json": {
        "type": "object",
        "properties": {"order_id": {"type": "string"}},
        "required": ["order_id"]
      }}
    }
  ]
}

resp = brt.converse(
  modelId=MODEL_ID,
  messages=[{"role": "user", "content": [{"text": "Where is order A-4471?"}]}],
  toolConfig=tool_config,
)
print(resp["stopReason"]) # -> "tool_use"

```

When *stopReason* is **tool_use**, the response contains a *toolUse* block with a *toolUseId*, the tool *name*, and an *input* object matching your schema. That's the model saying: "please run this for me."

2.3 A minimal agent loop from scratch

Here is the whole idea of an agent in ~25 lines: call the model, if it asked for a tool run it and feed the result back, repeat until it answers — with a hard iteration cap so it can't run away.

```
def run_tool(name, args):
    if name == "get_order_status":
        # real impl would query a DB / REST API
        return {"order_id": args["order_id"], "status": "shipped",
                "eta": "2 days"}
    return {"error": f"unknown tool {name}"}

def agent(user_text, max_steps=6):
    messages = [{"role": "user", "content": [{"text": user_text}]}]
    for _ in range(max_steps):
        # cap = no runaway loop
        resp = brt.converse(modelId=MODEL_ID, messages=messages,
                            toolConfig=tool_config)
        out = resp["output"]["message"]
        messages.append(out)
        # remember what model said
        if resp["stopReason"] != "tool_use":
            return out["content"][0]["text"]
        # final answer
        # else: run every requested tool, return results
        results = []
        for block in out["content"]:
            if "toolUse" in block:
                tu = block["toolUse"]
                result = run_tool(tu["name"], tu["input"])
                results.append({"toolResult": {
                    "toolUseId": tu["toolUseId"],
                    "content": [{"json": result}]}})
        messages.append({"role": "user", "content": results})
    return "Stopped: step budget exhausted."
```

This is the seed of every framework

LangGraph, CrewAI and Strands all generate a more sophisticated version of this loop — with planning, memory, retries and parallel tools — so you don't maintain it by hand. But when something misbehaves in production, this is the picture in your head that lets you debug it.

2.4 Build a tiny MCP server + client

Now we make our tool reusable by *any* MCP host. With the Python SDK's FastMCP, a server is mostly a decorator. The docstring becomes the tool description the model reads.

```
# file: order_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("orders")

@mcp.tool()
def get_order_status(order_id: str) -> dict:
    """Look up the current status of a customer order by id."""
    return {"order_id": order_id, "status": "shipped", "eta": "2 days"}

if __name__ == "__main__":
    mcp.run(transport="stdio") # local subprocess transport
```

To use it from Claude Desktop you add the server to its MCP config so the host launches it and discovers the tool automatically:

```
{
  "mcpServers": {
    "orders": {
      "command": "python",
      "args": ["/abs/path/order_server.py"]
    }
  }
}
```

The payoff in one sentence

You wrote *get_order_status* once. The from-scratch agent, Claude Desktop, your IDE assistant, and later an AgentCore-hosted agent can all call it — no per-host integration. That is the $N \times M \rightarrow N + M$ win from Part 0.

2.5 Folder structure & tool-design rules

```
my-agent/
agent/
  __init__.py
  loop.py           # the orchestration loop / framework wiring
  prompts.py       # system prompt, tool descriptions
tools/
  orders.py        # plain functions
  mcp_server.py    # FastMCP wrapper exposing tools/
memory/
  store.py         # short/long term read+write
evals/
  cases.jsonl      # input -> expected behaviour
  run_evals.py
config/
  settings.yaml    # model id, region, limits, flags
requirements.txt
README.md
```

Tool-design rules that prevent 80% of agent bugs

- **Idempotent writes.** Give write tools a client-supplied idempotency key so a retried *issue_refund* doesn't refund twice.
- **Tight schemas.** Constrain inputs (enums, required fields, formats) so the model can't pass garbage.
- **Structured, bounded output.** Return small JSON, not giant blobs — every token re-enters the context and costs money.
- **Explicit errors.** Return `{"error": ...}` the model can reason about; don't throw raw stack traces into context.
- **Timeouts + retries with backoff** on every external call.
- **Least privilege.** The tool's credentials should allow exactly that one action, nothing more.

Phase 3 · AWS Bedrock AgentCore

Everything so far ran on your laptop. Phase 3 is about running it *securely, at scale, in production* on AWS — first the building blocks, then AgentCore, which packages them for agents.

3.1 Where plain Bedrock fits

Amazon Bedrock is the managed model layer — the **brain supplier**. Before AgentCore you already get:

- **Models on demand** (Claude, Nova, Llama, Mistral...) via one IAM-authenticated API.
- **Converse API** — the uniform message + tool-calling interface you used in Phase 2.
- **Knowledge Bases** — managed RAG: point at S3, it chunks, embeds, stores in a vector DB, and retrieves.
- **Guardrails** — content filters, denied topics, and PII redaction applied to inputs and outputs.
- **(Legacy) Bedrock Agents** — the earlier, more prescriptive managed-agent feature; AgentCore is the newer, framework-agnostic platform.

The split to remember

Bedrock = the model + RAG + guardrails (the brain and its knowledge). **AgentCore** = the runtime, tools, memory, identity and observability that turn a brain into a deployed, governed agent. They're complementary; AgentCore calls Bedrock for the thinking.

3.2 Mapping agent concepts onto AWS building blocks

If you built an agent the manual way (no AgentCore), this is roughly how the pieces land on AWS. It's worth knowing because AgentCore abstracts several of these for you.

Agent concept	Manual AWS service	Role
The brain	Amazon Bedrock	LLM inference + RAG + guardrails
The agent process	Lambda / Fargate / EC2	Runs your loop code
Tools (your APIs)	API Gateway + Lambda	Callable functions/endpoints
Deterministic workflow	Step Functions	Orchestrates fixed multi-step processes
Memory / state	DynamoDB, S3, OpenSearch	Short/long-term + vector store
Permissions	IAM roles & policies	Who/what can call what

Agent concept	Manual AWS service	Role
Network isolation	VPC, subnets, SGs, endpoints	Keep traffic private
Secrets	Secrets Manager	API keys, tokens
Logs/metrics/traces	CloudWatch, X-Ray	Observe & debug

Step Functions vs the agent loop

Use the **agent loop** when the path is dynamic and the model should decide the next step. Use **Step Functions** for the deterministic spine — fixed, auditable steps with retries and human-approval gates. Real systems combine them: a state machine that calls an agent for the fuzzy step.

3.3 AgentCore at a glance

AgentCore is a modular platform — you can adopt one service or all of them, with any framework and any model. The services:

Service	One-line job
Runtime	Serverless, session-isolated place where your agent code executes
Gateway	Turns APIs / Lambda / OpenAPI / existing MCP servers into secured MCP tools
Memory	Managed short-term and long-term memory for agents
Identity	Agent identity + inbound/outbound auth with your IdP and 3rd-party services
Observability	OpenTelemetry traces, step-by-step execution view, CloudWatch metrics
Evaluations	Automated, data-driven quality scoring of agent runs
Policy	Deterministic guardrails on tool calls, authored in natural language or Cedar
Code Interpreter	Sandboxed environment for the agent to run code
Browser	Managed cloud browser for the agent to navigate the web

3.4 Runtime — where agents actually execute

Kid version. Runtime is a private, locked workshop that AWS spins up just for one job, then sanitises when the job's done — so two customers' jobs never share a bench.

Engineer version. Runtime is a serverless, framework-agnostic host for agent (or tool/MCP) containers. Key mechanics worth committing to memory:

- **Session isolation.** Each session runs in a dedicated microVM with isolated CPU, memory and filesystem. On termination the microVM is destroyed and memory sanitised — no cross-session leakage.
- **Sessions** are keyed by a *runtimeSessionId*, preserve context across turns, can run up to **8 hours**, and terminate after ~15 min idle. Session state is ephemeral — use the Memory service for durable context.
- **Versions & endpoints.** Every config change creates an immutable version; a stable *DEFAULT* endpoint (or custom dev/test/prod endpoints) points at a version, enabling zero-downtime updates and rollbacks.
- **Protocols.** HTTP (request/response), MCP (host tools/servers), and A2A (agent-to-agent). Streaming and WebSocket are supported for responsive UX.
- **Long-running & async** work is supported via background tasks and a */ping* health signal.

The fastest way in is the starter toolkit + SDK. You wrap your existing agent (any framework) with an entrypoint, then configure/launch/invoke from the CLI:

```
# my_agent.py (works with Strands, LangGraph, CrewAI inside the function)
from bedrock_agentcore import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@app.entrypoint
def invoke(payload):
    user_text = payload["prompt"]
    answer = my_existing_agent(user_text)  # your Phase-2 loop / framework
    return {"result": answer}

if __name__ == "__main__":
    app.run()
```

```
# CLI: containerise -> push to ECR -> create Runtime + DEFAULT endpoint
agentcore configure --entrypoint my_agent.py
agentcore launch
agentcore invoke '{"prompt": "Where is order A-4471?"}'
```

Mind shift from Lambda

A Lambda is short-lived and stateless per invoke. A Runtime *session* is a long-lived, isolated context that can think for minutes-to-hours and hold a conversation — purpose-built for agentic, multi-turn, tool-using work.

3.5 Gateway — turn APIs into MCP tools

Kid version. Gateway is a universal power strip: plug in your existing APIs and Lambdas on one side, and every MCP-speaking agent gets clean, labelled sockets on the other — with a security guard checking IDs in both directions.

Engineer version. Gateway is a fully-managed MCP server fronting your capabilities. You register **targets**; Gateway auto-generates MCP tools and handles protocol translation, tool discovery, and two-way auth.

Target type	What you point it at	Auth to the backend
Lambda	An AWS Lambda function	IAM
OpenAPI	An OpenAPI/Swagger spec for a REST API	OAuth / API key (egress auth)
Smithy	An AWS Smithy model	IAM
MCP server	An existing MCP server to aggregate	Pass-through

- **Ingress auth** — who may call the Gateway (IAM SigV4 or OAuth bearer tokens from your IdP).
- **Egress auth** — how the Gateway authenticates to the backend API (IAM, OAuth, or API key), managed by AgentCore Identity so secrets never touch your agent code.
- **Aggregation** — group several task-specific MCP servers behind one managed Gateway endpoint.

Typical registration flow (conceptual): upload your OpenAPI spec to S3, create a Gateway, add a target referencing the spec, attach an egress credential provider, and Gateway publishes each operation as an MCP tool at a single endpoint URL your agent points its MCP client at.

3.6 Memory · 3.7 Identity · 3.8 Observability, Evaluations, Policy

Memory

Managed **short-term** memory (multi-turn conversation context) and **long-term** memory (facts, preferences, summaries that persist across sessions and can be shared between agents). It removes the DynamoDB/vector-store plumbing you'd otherwise build, and lets agents learn from past interactions with control over what's retained.

Identity

Agent identity and access management compatible with existing IdPs (Cognito, Okta, Microsoft Entra ID, Auth0). It powers both inbound auth (validating who invokes the agent) and outbound auth (letting the agent act against Slack, GitHub, Salesforce, AWS, etc.) in two modes: **user-delegated** (on behalf of the signed-in user) and **autonomous** (the agent's own service credentials). Credentials are vaulted, never exposed in code or logs.

Observability

Emits standardised **OpenTelemetry** traces and integrates with CloudWatch, giving a step-by-step view of the agent's trajectory: which tools ran, with what inputs/outputs, latency per step, and where it failed. This is how you debug a non-deterministic agent — you read its trace, not just its logs.

Evaluations & Policy

- **Evaluations** — automated scoring of sessions/traces/spans against quality criteria, so you can measure regressions before and after deploy. Results flow into Observability.

- **Policy** — deterministic rules (natural language or AWS Cedar) that intercept every tool call at the Gateway and allow/deny based on which tool, what action, and under what conditions — guardrails the model cannot talk its way around.

3.9 How AgentCore hosts frameworks & MCP

Runtime doesn't impose a framework. Your container can wrap a Strands, LangGraph, CrewAI, LlamaIndex, Google ADK or OpenAI-Agents agent, calling any model in or outside Bedrock. The agent reaches tools by pointing its MCP client at a Gateway endpoint; it persists state via Memory; it authenticates via Identity; and it streams traces to Observability. Each service is opt-in — you can use Gateway alone, or Runtime alone, and grow into the rest.

3.10 MCP + AgentCore + Claude working together

Because AgentCore speaks MCP throughout, a coding assistant like Claude (in an IDE or the desktop app) becomes a genuine build-and-deploy partner. There is even an AWS-published AgentCore MCP server that gives an assistant first-class knowledge of AgentCore APIs. A realistic loop:

- 1 **Refactor.** You ask the assistant to wrap your local Phase-2 agent with the AgentCore endpoint and a *requirements.txt* — turning laptop code into a deployable Runtime container.
- 2 **Expose tools.** It generates/validates an OpenAPI spec for your REST API and scripts the Gateway target + egress credential so the API becomes an MCP tool.
- 3 **Wire credentials.** It configures Identity (inbound IdP, outbound providers) and writes least-privilege IAM.
- 4 **Deploy.** It runs *agentcore configure / launch*, watches the build, and reports the endpoint ARN.
- 5 **Test in natural language.** You say "check order A-4471"; it calls *agentcore invoke*, reads the Observability trace, and tells you which tool ran and why — iterating with you.

Production checklists (per component)

Runtime: iteration cap, session timeout, custom prod endpoint, rollback plan, async */ping* if long-running. **Gateway:** ingress auth on, egress creds in Identity, least-privilege target IAM, Policy rules on write tools. **Memory:** decide retention + what's long-term, PII handling. **Identity:** user-delegated vs autonomous chosen per tool. **Observability:** tracing enabled, alarms on error rate/latency/cost, Evaluations in CI.

Phase 4 · End-to-end examples

Four realistic scenarios. For each: the data flow, how security/permissions are set, and how you observe and debug it. These compose the Phase-3 services into whole systems.

A. Customer support agent

Scenario. A web chat where customers ask "where's my order?" and "can I get a refund?". The brain is Claude on Bedrock; an order-status REST API and a refund Lambda are exposed as tools via Gateway; conversation + customer preferences live in Memory.

Data flow

- 1 Customer sends a message; the web app attaches the customer's OAuth bearer token and a stable *session id*.
- 2 Request hits the Runtime endpoint; **Identity inbound auth** validates the token before the agent runs.
- 3 Agent (in Runtime) loads recent context from **Memory**, calls Bedrock, which emits a *get_order_status* tool-use.
- 4 Agent's MCP client calls **Gateway**; Gateway uses **egress auth** to call the order REST API and returns a clean MCP tool result.
- 5 For a refund, **Policy** checks the call (e.g. amount $\leq$ limit, customer verified) before the refund Lambda runs; otherwise it escalates to a human.
- 6 Bedrock composes the answer; the agent writes salient facts to long-term **Memory** and streams the reply back. Every step is emitted to **Observability**.

Security & permissions

- Inbound: OAuth via your IdP; only authenticated customers reach the agent.
- Outbound: order API and refund creds vaulted in Identity; the agent never sees raw secrets.
- Least privilege: refund tool's IAM allows only *issue_refund*; idempotency key prevents double refunds.
- Policy + human-in-the-loop gate all money-moving actions; Guardrails redact PII.

Observe & debug

If a customer reports a wrong refund, open the session trace in Observability: you see the exact tool inputs, the Policy decision, and the model's reasoning step that triggered it — far faster than grepping logs. Alarms watch error rate, p95 latency, and token/tool cost per session.

B. Internal ops assistant (with Step Functions)

Scenario. "Onboard new hire Asha" — a request that must always do the same governed steps: create accounts, assign a laptop, file an IT ticket, notify the manager. Here the deterministic spine is a **Step Functions** state machine; the agent handles only the fuzzy

step (drafting a personalised welcome + deciding access groups from the role description).

Data flow & why this shape

- Step Functions runs the fixed, auditable sequence with built-in retries and an approval gate before granting elevated access.
- One state invokes the AgentCore Runtime agent to reason over the role and propose access groups; the proposal returns to the state machine.
- A manager approval task (human-in-the-loop) must pass before the provisioning tools fire via Gateway.
- **Lesson:** don't let the model freelance compliance-critical workflows — wrap the determinism in Step Functions and call the agent only where judgement is needed.

Security & observe

Each provisioning tool has its own least-privilege role; Identity supplies per-service credentials; Step Functions' execution history plus AgentCore traces give a complete, replayable audit trail — exactly what an internal-ops/compliance review wants.

C. Coding / research assistant over MCP

Scenario. An IDE assistant that searches internal docs, runs snippets, and opens PRs. Tools are exposed as MCP: a docs-search server, a code-exec capability via AgentCore **Code Interpreter**, and a GitHub integration via **Identity** outbound (user-delegated, so it acts as *you*).

- MCP means the same servers work in the IDE during development and in a Runtime-hosted agent in CI.
- Code Interpreter sandboxes execution so generated code can't touch the host.
- User-delegated GitHub auth keeps actions attributable to the real engineer — good for audit.
- Observability traces show which doc chunks and which code runs produced a suggestion — vital for trust.

D. Document-intelligence agent (RAG + Knowledge Bases)

Scenario. "Summarise our obligations under the Acme contract and flag renewal risk." The agent answers from your own documents using a Bedrock **Knowledge Base** for retrieval, with a calculator tool for dates/amounts and Memory for the running analysis.

- Documents land in S3; the Knowledge Base chunks, embeds and indexes them — managed RAG, no vector DB to run.
- Each turn: retrieve relevant chunks → ground the model on them → cite sources back to the user.
- **Guardrails** block the model from answering outside the retrieved context (reduces hallucination) and redact PII.

- Debugging a bad answer = inspecting the retrieved chunks in the trace; usually it's a retrieval problem, not a model problem.

The pattern across all four

Same skeleton every time: **request → inbound auth → agent loop (Bedrock brain) → tools via Gateway/MCP with egress auth + Policy → Memory read/write → response**, with Observability on every step. Once you see it once, you see it everywhere.

Recipes · copy-and-keep checklists

Recipe 1 — POC Python agent → production AgentCore agent

- 1 Freeze the loop: add a max-steps cap, per-tool timeouts, and a graceful fallback answer.
- 2 Externalise config (model id, region, limits) into *settings.yaml*; pin dependencies in *requirements.txt*.
- 3 Wrap the agent with the AgentCore *@app.entrypoint* so it takes a payload and returns a result.
- 4 Move tools behind Gateway (Lambda/OpenAPI targets) instead of in-process functions.
- 5 Replace ad-hoc memory with the Memory service; decide what is long-term.
- 6 Add Identity: inbound IdP auth + outbound credential providers (no secrets in code).
- 7 *agentcore configure* → *launch*; create a **prod** endpoint distinct from dev.
- 8 Turn on Observability; add alarms on error rate, latency, and cost.
- 9 Write evals (input → expected behaviour); gate deploys on them; roll out via endpoint canary.

Recipe 2 — expose a REST API as a secure tool via Gateway

- 1 Write/clean an OpenAPI spec; give each operation a clear summary — that text becomes the tool description.
- 2 Upload the spec to S3.
- 3 Create a Gateway; add an **OpenAPI target** referencing the spec.
- 4 Configure **egress auth** (OAuth/API key) in Identity for the backend; store nothing in the agent.
- 5 Set **ingress auth** (IAM or OAuth) so only your agents can call the Gateway.
- 6 Add **Policy** rules for any write/destructive operations.
- 7 Point the agent's MCP client at the Gateway endpoint; confirm the tool appears in discovery and test one call.

Recipe 3 — wire an MCP client (Claude Desktop / IDE) to help refactor & deploy

- 1 Install the AWS AgentCore MCP server (and your own tool servers) into the assistant's MCP config.
- 2 Ask it to refactor your local agent to the AgentCore entrypoint + requirements.
- 3 Have it generate the Gateway target script and least-privilege IAM for your API.
- 4 Let it run *agentcore launch* and report the endpoint ARN.
- 5 Drive end-to-end tests in plain language; have it read the trace and propose fixes.

Pre-flight · questions before you ship

Security

- Who can invoke the agent, and is inbound auth enforced at the Runtime endpoint?
- Does every tool run with least-privilege credentials, vaulted in Identity (never in code/logs)?
- Are write/destructive actions gated by Policy and/or human approval, and idempotent?
- Are Guardrails redacting PII and blocking disallowed topics on input and output?
- Is traffic to private resources kept inside a VPC where required?

Reliability

- Is there an iteration cap and a sane fallback when the budget is hit?
- Do all external calls have timeouts + retries with backoff?
- Is there a prod endpoint separate from dev, with versioning and a tested rollback?
- What happens on a tool outage — does the agent degrade gracefully?

Cost

- What is the expected token + tool-call cost per session, and is there an alarm on it?
- Are tool outputs bounded so context doesn't balloon each turn?
- Is the right model used for the job (cheaper model for simple steps)?
- Are long sessions and async work bounded so Runtime time doesn't run away?

Evaluation & safety

- Do you have an eval set of representative cases with expected behaviour?
- Do deploys run evals and block on regressions?
- Are you canarying changes via endpoints and watching Observability before full rollout?
- For ambiguous or high-impact actions, is a human reviewing before the agent commits?

Best practices, pitfalls & glossary

Top pitfalls (and the fix)

Pitfall	Why it bites	Fix
No iteration cap	Runaway loops, surprise bills	max#8209;steps + fallback answer
Vague tool descriptions	Model picks the wrong tool	Crisp name + description + tight schema
Non#8209;idem potent writes	Double refunds on retry	Idempotency keys
Huge tool outputs	Context bloat & cost	Return small, structured JSON
Secrets in code	Leak risk; fails audit	Identity / Secrets Manager
Logs#8209;only debugging	Can't follow agent reasoning	Use OTEL traces / Observability
Model runs compliance flow	Unauditable, unsafe	Step Functions + Policy + human gate
No evals	Silent regressions	Eval set gating deploys

Glossary

- **Agent** — an LLM placed in a loop that can call tools and act toward a goal.
- **Tool / tool-use / tool-result** — a callable function; the model's request to call it; the value returned.
- **MCP** — Model Context Protocol; the open standard (server/client/host) for exposing tools and data to agents.
- **Converse API** — Bedrock's uniform message + tool-calling interface across models.
- **Runtime / Gateway / Memory / Identity / Observability** — AgentCore services for running, tool-wiring, remembering, authenticating, and watching agents.
- **Ingress / egress auth** — who may call the agent/Gateway; how the Gateway authenticates to a backend.
- **Session / microVM** — an isolated execution context (its own CPU/mem/fs) for one conversation.
- **RAG / Knowledge Base** — retrieval-augmented generation; Bedrock's managed retrieval over your documents.
- **Guardrails** — content filters, denied topics, and PII redaction on model I/O.
- **A2A** — agent-to-agent protocol for multi-agent communication.
- **Cedar** — AWS's open policy language used by AgentCore Policy for deterministic rules.

Where we are & what's next

You've now got the full arc: concepts → coding → AgentCore → four end-to-end examples, plus recipes and a pre-flight list. Read it, mark anything fuzzy, and bring those spots back — the natural next step is to actually build Example A together, code by code, on your AWS account.

Sources: AWS Bedrock AgentCore developer guide and AWS blog — Overview, Runtime “How it works”, Gateway, and FAQs (docs.aws.amazon.com/bedrock-agentcore, aws.amazon.com/bedrock/agentcore). Verified May 2026.